

Explicação Técnica do Código Tuning.java - FTC PedroPathing

Este documento fornece uma explicação detalhada, em nível universitário, do arquivo Tuning.java, utilizado no framework PedroPathing para FIRST Tech Challenge (FTC). O objetivo do código é permitir a calibração, validação e análise do sistema de navegação autônoma do robô.

1. Conceito Geral do Tuning.java

O arquivo Tuning.java implementa um OpMode especial de calibração. Ele não executa uma tarefa de jogo, mas fornece um ambiente controlado para testar odometria, controle PID, modelagem de trajetória e parâmetros físicos do robô. Em robótica móvel, esse tipo de software é considerado um banco de testes.

2. Arquitetura de Seleção de Testes

- Localization Test: valida posição (x, y) e orientação (heading).
- Forward/Lateral/Turn Tuners: calibram fatores de escala de movimento.
- Automatic Tuners: ajustam limites de velocidade e aceleração.
- Manual Tuners: permitem ajuste fino de controladores PIDF.
- Trajectory Tests (Line, Triangle, Circle): validam seguimento de caminhos.

3. Localization Test

O Localization Test atualiza continuamente o Follower e exibe a pose do robô. A pose é representada como (x, y, θ), sendo θ o ângulo de orientação. Este teste é fundamental para verificar a consistência da odometria e da IMU.

4. Forward, Lateral e Turn Tuning

Os tuners de deslocamento medem discrepâncias entre movimento real e estimado. Eles produzem multiplicadores de calibração para corrigir erros sistemáticos de encoder. O Turn Tuner calibra a conversão entre ticks de encoder e rotação angular do robô.

5. Testes de Trajetória (Line, Triangle, Circle)

Os testes geométricos avaliam o comportamento do controlador em trajetórias não-lineares. O teste circular mede erro centrípeto, o triângulo mede fechamento de loop, e a linha mede estabilidade de controle longitudinal. Esses testes são usados em robótica industrial e veículos autônomos.

6. Follower e Controle de Movimento

O Follower implementa algoritmos de controle de trajetória (semelhante a Pure Pursuit ou MPC). Ele calcula erros de posição, velocidade e orientação, e gera comandos para os motores. O método `update()` executa o loop de controle em tempo real.

7. Visualização e Telemetria

O código utiliza módulos de desenho para exibir a trajetória e o histórico de movimento. Essa técnica é equivalente a ferramentas de debug utilizadas em robôs móveis e drones.

8. Importância Acadêmica e Aplicações

Sistemas como `Tuning.java` são equivalentes a frameworks de calibração em robótica acadêmica. Eles permitem validação experimental, análise de erro, identificação de parâmetros e estudo de controle. Esse tipo de abordagem é comum em pesquisa em robótica móvel, SLAM e veículos autônomos.

Conclusão

O `Tuning.java` é uma ferramenta crítica para equipes FTC avançadas, pois fornece infraestrutura para calibração científica do robô. Para equipes iniciantes, recomenda-se compreender odometria básica e controle PID antes de utilizar este sistema completo.